PAC_MAC: Fine-Grained Memory Access Control for Inter-VM Communication

1st Giuseppe Alfieri
Università degli studi di Napoli Federico II
Napoli,Italia
alfierigiuseppe540@gmail.com

2nd Domenico Coppola
Università degli studi di Napoli Federico II
Napoli,Italia
dom.cop@libero.it

I. ABSTRACT

The adoption of virtualization in mixed-criticality systems enables strong isolation between domains sharing the same hardware platform. However, the de facto mechanism for inter-VM communication, Virtio, introduces a critical trade-off between performance and isolation: backends typically require unrestricted access to guest memory to achieve high performance, resulting in low isolation guarantees. Existing mitigations, such as Xen grant tables or bounce buffers, either provide coarse-grained control or impose significant performance penalties, making them unsuitable for safety-critical deployments.

We argue that secure and efficient inter-VM communication demands hardware-enforced memory isolation. To this end, we propose PAC_MAC, a hardware-assisted mechanism that enforces fine-grained memory mediation for Virtio backends. Our prototype, implemented on a Xilinx ZCU104 SoC running Jailhouse, combines backend with a custom FPGA module that intercepts every backend memory transaction. Unauthorized requests are blocked in hardware, while authorized requests are transparently forwarded to the DDR controller. PAC_MAC thus prevents malicious or faulty backends from violating isolation without requiring guest modifications.

II. INTRODUCTION

Context. Mixed-Criticality Systems (MCS) integrate applications with different levels of criticality on the same hardware platform. This integration aims to reduce costs, improve resource utilization, and enable flexible designs. To be safe, MCS requires strong spatial and temporal isolation between domains. Virtualization has recently emerged as a key enabling technology for this paradigm, with hypervisors enforcing hardware isolation among virtual machines (VMs). In particular, Static Partitioning Hypervisors (SPHs), such as Bao and Jailhouse, are well suited to MCS because they assign dedicated processor cores to VMs, providing strong timing guarantees and avoiding the complexity of general-purpose hypervisors like KVM or Xen.

Addressed problem. Even with SPHs, challenges arise when multiple VMs must share access to physical devices, since the number of peripherals is often smaller than the number of isolated domains. To enable device sharing, the de facto mechanism is VirtIO, which slits the device driver into a frontend (in the VM) and a backend (in the hypervisor or a dedicated service VM). While widely used, this design conflicts with the isolation requirements of MCS since the VirtIO backend assumes unrestricted access to the guest's memory, in contrast with the principle of spatial isolation.

Limitations of Existing solutions. Existing software-based mechanisms attempt to mitigate these issues, limiting the privileges of the service VM over the front-end VM. For example, grant tables in Xen allow controlled page-level sharing, and bounce buffers restrict DMA operations to specific memory areas. However, grant tables lack fine-grained control, and bounce buffers introduce performance overheads. As a result, there is still no solution that provides both security and efficiency for inter-VM communication in MCS. So, in this paper, we try to address the limitations of existing software-based solutions for secure inter-VM communication in Mixed-Criticality Systems. Our goal is to provide fine-grained and deterministic control of memory accesses performed by VirtIO backends, without affecting efficiency.

PAC-MAC Approach. We introduce PAC_MAC, a custom hardware component that validates memory transactions originating from the VirtIO backend. PAC_MAC is able to intercept and validate memory requests issued by the physical core (`Core-Trans`) of the service VM, as well as those performed by the peripheral (i.e., `DMA-Trans`) assigned to the service VM. For each request, PAC_MAC checks whether the target address falls within an authorized range. Authorized requests are forwarded to the memory controller, while unauthorized ones are blocked in hardware. By shifting access control to hardware, we believe PAC_MAC can remove the need for unrestricted backend access and avoid the performance penalties of bounce buffers. So, we conducted the following empirical evaluation. We integrated PAC_MAC into a prototype system based on the Jailhouse hypervisor running on a Xilinx ZCU104 SoC. In this setup, we created a bare-metal cell (i.e., the service VM), and we assigned to it the system DMA, in order to show empirical evidence of the overhead

of PAC_MAC, as well as its effectiveness in enforcing access checks for both kind of memory transactions (`DMA-Trans` and `Core-Trans`).

Contributions. This paper makes the following contributions:

- **Hardware-based solution.** We propose PAC_MAC, a hardware mechanism that transparently enforces fine-grained memory access control for VirtIO backends.
- **Prototype implementation.** We integrate PAC_MAC into Jailhouse on a Xilinx ZCU104 platform, demonstrating feasibility in an SPH-based MCS setup.
- **Evaluation.** We provide an experimental analysis to investigate the PAC_MAC effectiveness and overhead.

Our evaluation shows that PAC_MAC effectively prevents unauthorized memory accesses. Furthermore this work will show that going through FPGA instead of directly accessing memory causes an increase in time with respect to the number of transfers with a slope 9 times greater. This demonstrates the extreme difficulty in finding a perfect trade-off between security and performance: whenever one side is strengthened, the other tends to weaken. In fact, increasing security usually introduces additional layers of checks, verifications, or encryption mechanisms, which inevitably slow down processes and require more computational resources.

III. BACKGROUND AND MOTIVATIONS

We provide an overview of VIRTIO's operation and related security issues, as well as a technical overview of MPSoC FPGA devices, with a focus on models developed by Xilinx. To conclude, we introduce the unique features of the Jailhouse hypervisor.

MPSoC FPGA devices. Using Xilinx terminology, MPSoC FPGA devices are also referred to as PS-PL devices, in reference to the two domains they are composed of: the Processing System (PS) and the Programmable Logic (PL). The Processing System is the part of the SoC that includes the processor, memory subsystems, standard peripherals (e.g., UART, SPI, I2C), interrupt controllers (e.g., GIC), DMA, timers, and much more. The Programmable Logic section corresponds to the FPGA area integrated into the SoC. Thanks to the availability of reconfigurable hardware resources such as LUTs, flip-flops, DSP blocks, and BRAM, the PL allows for the implementation of custom digital circuits, ranging from simple custom peripherals to complex systems. The PS and PL communicate via interfaces based on the AXI (Advanced eXtensible Interface) protocol [7], a high-performance bus standard developed by ARM. AXI supports multiple independent channels for data and command transfer, allowing for simultaneous read and write operations. The main variants include:

1) **AXI4-Full:** for high-bandwidth data transfers between master and slave.
2) **AXI4-Lite:** for control register accesses, with a simplified protocol and low overhead.
3) **AXI4-Stream:**for continuous data streams, ideal for audio/video streaming or high-speed data.

These interfaces ensure efficient and flexible communication between the PS and PL, allowing the programmable logic to integrate tightly with the SoC's processor and peripherals [4] [7].

Jailhouse Hypervisor. Jailhouse [1] is a static partitioning hypervisor developed by Siemens in 2013. This hypervisor makes it possible to create "cells," which are isolated partitions, each with dedicated hardware resources. When Jailhouse starts, Linux runs in the root cell, the first cell created, which acts as the control cell. From this cell, subsequent non-root cells (called "inmates") can be created, operating in a strictly confined environment, each with exclusive access to its assigned hardware resources. An inmate can run a lightweight operating system, a bare-metal application, or specialized code. This benefits from direct hardware control and strong isolation from other cells, a fundamental characteristic for the security and temporal predictability typical of real-time and embedded systems [1].

VIRTIO. requires the back-end to have the total visibility of the front-end memory. Communication between the two parties occurs through three data structures, plus two buffers renamed as InBuff and OutBuff which are necessary for data exchange.

1) **Descriptors pool:** Each descriptor contains the pointer to memory area where the other entity will find the data
2) **Available queues:** Used by front-end to push ID of descriptor added in Descriptors pool
3) **Used queues:** Used by back-end to report the consumation of request
4) **OutBuff:** Input to the back-end
5) **InBuff:** Output from the back-end

When the front-end wants to send a write/read request to back-end, it writes the data in OutBuff, pushes the descriptor in Descriptors pool structure and associated ID in Available Queues. At that point the backend has everything it needs to start the processing, which culminates in writing the data in OutBuff and push of consumed descriptor ID in used queues. [11]

VIRTIO security. To understand why secure communication in VIRTIO is necessary, consider the following real-world scenario example:

1) A front-end VM, which runs a real-time control application (for example, managing an industrial robot or drone), where sensitive data is collected from sensors and cameras.
2) A back-end VM, which is supposed to forward packets to the network or hardware devices.

Assuming that in a certain point the back-end becomes compromised. Possible attacks caused by this dangerous situation are:

- Real-time memory scraping : The backend constantly reads shared buffers in the frontend's RAM to extract sensitive data (video streams, actuator positions, security parameters). Because access is direct, the attacker obtains the information without introducing any perceptible delay.
- Data injection with immediate impact : The backend writes falsified values to the memory buffers used for real-time control calculations.

IV. Problem Statement

A. Threat Model

In this scenario, the front-end is considered the trusted entity, while the back-end represents the main untrusted component. The back-end runs on a privileged VM in the classical VIRTIO scheme and has extensive control over system resources, including the front-end's memory. Starting from the data flow diagram showing in [Figure 1] the VIRTIO security issues, a thread modeling process was used to identify the threats caused by untrusted back-end presence.

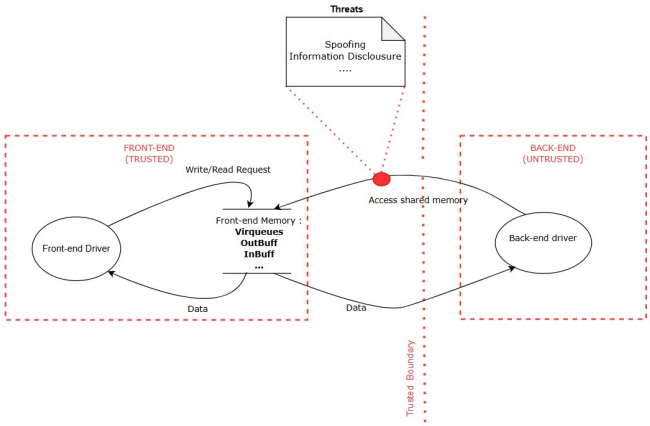


Fig. 1. Data flow diagram.

The presence of a malicious back-end can cause this critical issues :

1) **Spoofing.** : Since back-end, which executes on privileged VM, can send fake data to front-end, which might think it is authentic.
2) **Tampering.** : Back-end can send malformed data that changes the state of the Front-end to an unexpected state.
3) **Repudiation.** : Back-end can deny having modified the front-end memory.
4) **Information Disclosure.** : Backend has complete visibility of front-end memory, so sensitive data is exposed.

B. Software solutions and their limitations

This section discusses some possible software solutions to VIRTIO security issues and their limitations.

Grant tables. A significant example of a software solution addressing the limitations of VIRTIO is the Grant Table mechanism in the Xen hypervisor. Considering two VM (or domains in Xen terminology) called Dom0 (backend) and DomU (frontend). The communication flow includes the following phases:

1) DomU chooses a page to share, updates the grant table (with permissions and so on..) and sends a grant reference (numeric id) to Dom0
2) Dom0 invokes the hypercall , using the grant reference, to request memory mapping to Xen
3) Xen checks the validity of permissions in Grant table and if so allocates the required memory [8].

Bounce Buffers. Alternative to Grant Tables solution which is implemented using the Linux restricted Direct Memory Access (DMA) pool mechanism. All I/O traffic is redirected to an intermediary buffer (the bounce buffer). This prevents a backend from arbitrarily accessing a guest's private memory. In this way, the original buffers are copied into controlled and secure areas that are accessible to the backend. They are employed to address the issue of memory access granularity. Since the DMA controller operates at the granularity of entire pages, accessing a small portion of a page requires mapping the whole page. This can inadvertently expose critical or sensitive data that resides in other parts of the same page [9].

Software solutions, discussed above, mitigate many of threats presented in *Threat Model* section.

If Grant-tables mechanism is used the front-end no longer exposes all of its RAM to the back-end. Instead, the front-end explicitly grants the back-end access to specific memory pages, and only for as long as necessary.

If Bounce-Buffers mechanism is used the back-end no longer works on the front-end's "real" buffers, but on copies. This reduces the direct exposure of the front-end's RAM.

Unfortunately both solutions introduce some limitations: in the case of Grant Tables [8], due to memory being managed at the page granularity, and in the case of Bounce Buffers [9], because of the extra processor operations required to copy data through intermediate buffers.

V. PAC_MAC Overview

A. Design Goals

In the classic VIRTIO scheme, the backend has full visibility into the frontend's memory as shown in [Figure 2].

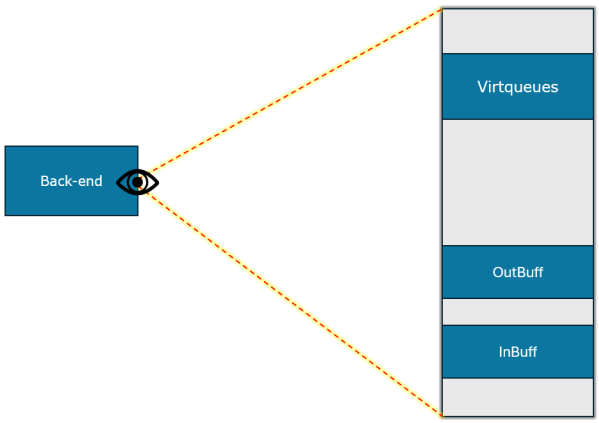


Fig. 2. VirtIo classic scheme.

Our goal is to interpose an FPGA component called PAC-MAC that forces the backend to have visibility only into the Virtqueues and the two buffers used for data exchange (called OutBuff and InBuff) [Figure 3]

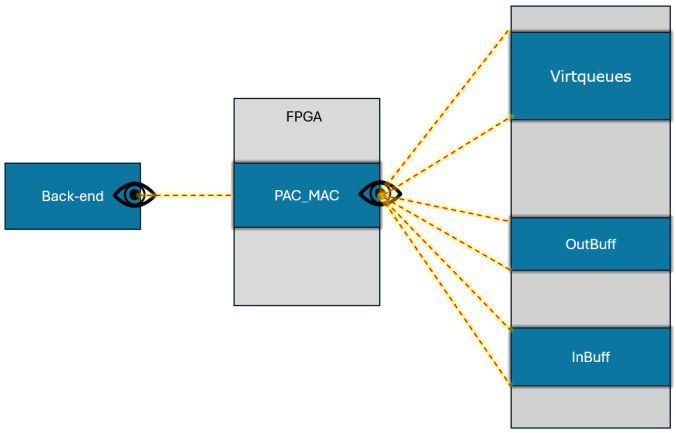


Fig. 3. Scheme with PAC-MAC.

The requirements analysis highlights the subsequent design requirements that must be met:

- **Functional Requirements** :
  1) Intercepting the read/write requests
  2) Control memory accesses
  3) Signal errors in case of invalid requests
- **Non-functional Requirements** :
  1) Security
  2) Robustness
  3) Standard Compliance (e.g AXI)

B. Transparent Design

This design is transparent to the master because it handles AXI transactions without affecting valid operations. The PAC-MAC module only intervenes for unauthorized transactions, generating the appropriate AXI error responses. This approach allows the system to maintain security and error handling while remaining non-intrusive and transparent to standard operations.

VI. Key techniques

A. Virtualization

The first step is to simulate a malicious back-end through virtualization, using Jailhouse as a static partitioning hypervisor. Jailhouse is a hypervisor that divides the hardware into isolated execution environments called cells. The **root cell** is the main partition, runs Linux, and is responsible for overall system control. Initially, it manages most hardware resources and configures additional cells. The cell that will simulate a malicious back-end, called `dma_cell` will have a DMA peripheral allocated to transfer bursts of valid or invalid data to the FPGA. It will therefore have several allocated memory areas, some dedicated to data transfer to the FPGA and others designed to contain executable code and allow possible (and optional) communication with the root cell.

B. Filtering

When a core or DMA inside a Jailhouse cell initiates a transfer to the FPGA, the PAC-MAC module within the FPGA filters the request based on the read or write address. Invalid requests are blocked with an AXI error response, while valid transactions pass through unaltered.

C. FPGA in the middle

The FPGA acts as an intermediate point between the PS/DMA and the memory peripherals, meaning it is located ”at the center” of the data flow. This positioning allows for transaction control and filtering without requiring changes to the master or peripheral software. By operating in this central location, the FPGA can protect the system (block unauthorized access, generate errors) and correctly route data to the appropriate destination, (i.e DRAM)

VII. Detailed Design

The final design integrates three main blocks, connected via an AXI SmartConnect interconnect. The master, represented by the PS, generates an AXI transaction via the `M_AXI_FPD_HP0` port. This transaction is redirected via the SmartConnect to the ATL (AXI Transaction Logic). The ATL generates the corresponding memory addresses (from 0x0 to 0x7FFFFFFF), which are analyzed by the PAC-MAC module. If the transaction is valid, the PAC-MAC component addresses the RAM (by prefixing 0x00 to the preceding 32 bits), allowing the transaction to pass through. Otherwise, it addresses its second slave interface (by prefixing 0x11 to the preceding 32 bits), to which a PAC-MAC submodule called SINK is connected. This SINK submodule is responsible for the error response to the AXI transaction.

- **ATL Address Translation Logic:** The `ATL_v1_0` IP converts the 40-bit PL addresses into effective 32-bit destination addresses (memory or peripheral). All AXI requests coming from the PS pass through the ATL without any filtering: the 8 most significant bits of the address are discarded. (`m_axi_awaddr<= s_axi_awaddr(31:0)`) (same for read address araddr), while burst, length, cache attributes, and handshake signals are propagated unchanged. [Figure 4] [5].
- **PAC_MAC:** Immediately after the ATL, the transactions reach the `PAC_MAC` block, which contains two components:

1) **FILTER:** it verifies that the starting address and the last address of each burst fall within fixed authorized ranges defined through an upper and lower bound (`WADDR_LOW/SUP and RADDR_LOW/SUP`). If the check is positive, it applies the prefix 00 to the 32 bits of the address and forwards the transaction to the `S00_AXI` port through the `M_AXI` channel. If it is negative, it applies the prefix 11 and routes the request to the `S01_AXI` port. [Figure 4].

2) **SINK:** it is connected to the `S01_AXI` port and acts as a "trash bin" for unauthorized transactions: it consumes the transmitted beats, replies with an error code, and does not access memory [Figure 4]. According to AMBA AXI specification the error response for writing can only be sent, by SINK module, at the end of burst. This requires SINK to ignore all burst beats and upon receiving WLAST=1, terminate the transaction with BRESP=SLVERR error. The same applies to reading, except that in this case is SINK itself signals the end of burst, so it can immediately respond with RLAST=1 and immediately send the RRESP=SLVERR error message. This module is mandatory because if the address is not valid, it is not enough to ignore the transaction. The AXI master can be blocked, and, in this regard, not all the architectures have a timeout enabled. So, the SINK module is responsible for completing the AXI transaction by responding with an AXI error (BRESP=SLVERR=10 in case of writing or RRESP=SLVERR="10" in case of reading) [7].

- **AXI SmartConnect:** The outputs of the ATL and the `PAC_MAC` are connected to `axi_smc` (SmartConnect) modules, which act as a crossbar between the masters (DMA, PS core, PL logic) and the slaves (DDR controller, sink). Thanks to these interconnects, valid transactions are directed to the DDR controller, while those marked as invalid are redirected to the SINK. The SmartConnect modules handle request arbitration, synchronize the different clock domains, and support scalability to additional masters or slaves. [Figure 4].

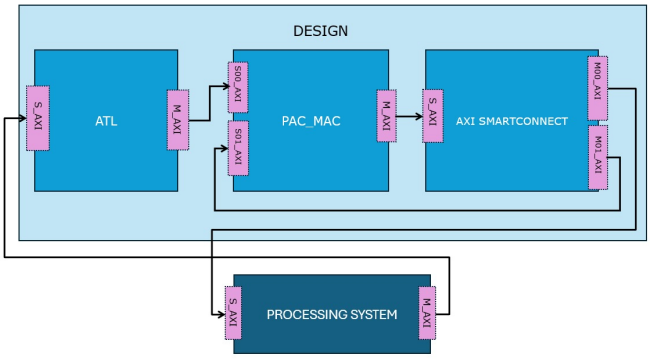


Fig. 4. Design hardware.

A. Discussion

Starting from the detailed design description shown in the last section, it's easy to understand the associated challenge faced for each component.

- **ATL** : faces the challenge of intercepting memory requests from the PS side.
- **PAC_MAC** : faces two challenges: the first is parsing write/read requests via the FILTER sub-module, and the second is blocking invalid transactions via the SINK sub-module.

All requirements are met: ATL translates addresses, FILTER allows only valid accesses, and SINK handles errors securely, ensuring AXI- compliance, robustness, and security.

VIII. IMPLEMENTATION

A. Detailed Jailhouse cell configuration

After loading the kernel module required to use Jailhouse (jailhouse.ko), the various hardware resources are partitioned by the Jailhouse hypervisor into two isolated partitions (referred to as "cells").

Root cell "ZynqMP-ZCU104": The root cell is the first cell that must be created in Jailhouse. It runs Linux and is primarily responsible for control functionalities. The configuration file required to enable it, called zynqmp-zcu104.cell, initially assigns all cores to the root cell (cpus=0xf). Subsequently, when one or more non-root cells are created, these cores will be removed from the root cell and assigned to them. In the configuration file, 16 MB of RAM (0x7f000000–0x7fffffff) is reserved for the hypervisor's code; the rest of the DDR is assigned to the root cell, with the exception of the regions reserved for inmates. The main MMIO peripherals are mapped starting from 0xfd000000 (about 3 MB). It includes an IVSHMEM device for sharing memory between VMs (not used directly in this experiment, but available for any future extensions).

Non-root cell "DMA-cell": It's assigned only a single core and has a memory region indicated as RAM that starts at address 0x50600000 with a size of 4 MB [Figure 5]], marked with appropriate flags to allow firmware loading and to contain executable code. It has two memory regions dedicated to data transfer to and from the FPGA, marked with `JAILHOUSE_MEM_DMA`:

- **Mem for DMA region** [Figure 5] : region of 2MB with starting address 0x50A00000. If you're not going through the FPGA, the FPD-DMA will write the data directly from the source buffer. This buffer can be allocated in RAM or within this region itself, as long as you're careful to keep it separate from the transfer's destination address. If the requests are sent to the hardware design on the FPGA, the FPGA will do the passthrough of the write/read operation if the request to/from this memory region is considered valid..

- **Mem for FPGA region** [Figure 5]: region of 2 MB with starting address 0x1050A00000. It's visible from the programmable logic, which allows for the interception of memory requests.

The non-root cell also has two regions marked with `JAILHOUSE_MEM_IO`.

- **FPD-DMA region** [Figure 5] : with starting address of 0xFD500000 and a size of 64 KB, allows direct access to the FPD-DMA registers [12].

- **CRF-APB region** [Figure 5] : with starting address of 0xFD1A0000 and a size of 4 KB, allows for enabling the clock for the FPD-DMA via a specific register mapped in this address range, called `FPD_DMA_REF_CTRL` [12].

It includes an optional 4 KB communication region at 0x80000000 for exchanging messages with the root (e.g., notifications or statistics). As for hardware interrupts, the cell only receives the necessary GICv2 lines (IRQ 140, 141). Line 156 must be added if you want to use the interrupt generated by the Channel 0 of FPD-DMA.

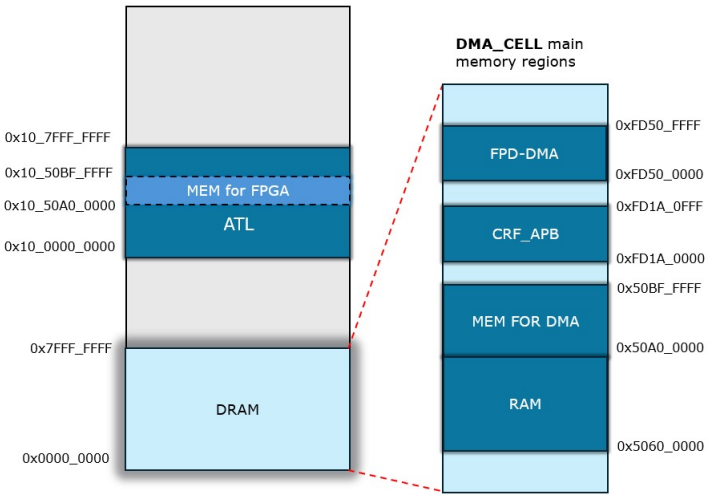


Fig. 5. Address ranges.

B. DMA cell firmware development

The bare metal firmware loaded into the `dma_cell` is responsible for configuring the FPD-DMA controller, managing the cache, and measuring performance. The main functions are described below.

- **D-cache management:** Since the FPD-DMA is not coherent with the CCI, it is necessary to ensure that the data present in the cache is coherent with memory before and after each transfer. In general, two approaches can be applied, not necessarily mutually exclusive. The first option is to manually perform appropriate cache operations by exploiting assembly instructions provided by ARM. The function: `dcache_flush_range(addr,size,choice);` executes precisely this task on a custom address range provided as input. It reads the `CTR_EL0` register to determine the cache line size, aligns the interval to multiples of this size, and iterates over the individual cache lines. Depending on the `choice` parameter, it performs three main operations:
  1) CLEAN (`DCACHE_CLEAN`): writes back the "dirty" cache lines within the specified address range to memory. The relevant assembly instruction is `dc cvac`, whose full name is "Data or unified Cache line Clean by VA to PoC". It ensures cache cleaning up to the Point of Coherency (PoC), i.e., the point where all masters (DMA, processor, etc.) have a coherent view of memory.
  2) INVALIDATE (`DCACHE_INVALIDATE`): invalidates the cache lines within the specified address range, using a similar approach as the previous case but with the `dc ivac` instruction ("Data or Unified Cache line Invalidate by VA to PoC"), which invalidates the cache up to the PoC.
  3) CLEAN+INVALIDATE (`DCACHE_CLEAN_INV ALIDATE`): performs both of the previous operations, using the `dc civac` instruction ("Data or unified Cache line Clean and Invalidate by VA to PoC").

  Another approach is to use the function provided by Jailhouse: `map_range(start,size,map_type)` which allows marking a specific memory region defined by starting address and size as either `MAP_CACHED` or `MAP_UNCACHED` depending on the third parameter. The two approaches are not mutually exclusive. In some cases, it may be mandatory to "declare" a memory region with `map_range` for instance, when those addresses are required to access peripheral registers while in other cases a region may be marked as `MAP_CACHED` and the previously described function can be used to manually operate on the cache.
- **DMA Configuration:** FPD-DMA available on PS-size of ZCU104 can operate in multiple modes. In our context "simple mode" was used and allows a contiguous block of data to be transferred from one address to another. The implemented function is : `gdma_config_simple_mode(GDMA_CH_ _BASE_ADDR,src_addr, dst_addr, length);` It receives as input the base address of FPD-DMA channel (there are seven available channels), the source and destination address, whose maximum supported size is 40 bits, and the number of words to transfer, starting from address `src_addr`. The conversion of length to bytes is done inside the function, by multiplying the number of words by 4. [2] [12]
- **Start of transfer:** The transfer is started by setting the bit 0 of `GDMA_CH_CTRL2` register to 1 through `gdma_start_transfer(GDMA_CH_BASE_ADDR)` function [2] [12].
- **Wait for transfer completation:** It is done with `gdma_poll_for_conversion(GDMA_CH_BASE_A DDR)` function. It controls the transfer state, polling on `GDMA_CH_STATUS` register, and gives 0 as output if the transfer ends successfully (`DONE_WITH_NO_ERROR` (i.e. 0x00)), and -1 otherwise (`DONE_WITH_ERROR` (i.e. 0x03)) [2] [12].
- **Enabling the clock:** At startup, the firmware calls `power_on_fpd_dma(GDMA_CH_BASE_ADDR)` function, which accesses to `FPD_DMA_REF_CTRL` register and enables the FPD-DMA clock. Without this

operation, changes to registers will have no effect [2] [12].

IX. Evaluation

The purpose of this section is to present the main analyses conducted on the system. The objective is to answer the following research questions.

- **RQ1** : **Significant Factors**. What factors have a statistically significant effect on transfer time?
- **RQ2** : **Impact of FPGA on DMA Transfer Time**. What is the impact of adopting PAC_MAC module on DMA transfer times?
- **RQ3** : **Impact of FPGA on CPU Transfer time**. What is the impact of adopting PAC_MAC module on CPU transfer times?
- **RQ4** : **Concurrent Access Analysis**. How do concurrent accesses from multiple masters affect the transfer times of memory requests when they are validated by PAC_MAC?

A. Metodology

To analyze transfer times and assess the impact of various factors, a statistical approach was adopted. The **workloads** used to perform the various experiments are described below:

- `gdma_stress_test(GDMA_CH_BASE_ADDR, src_addr,dst_addr,length,repetitions)` : This routine configures DMA channel, chosen with base address provided as input. It then performs `repetitions` transfers sequentially. The total time required is measured with `timer_get_ticks()` function (through reading of `CNTFRQ_EL0` register, which is a system register that contains the frequency of system timer) and converted into nanoseconds with `timer_ticks_to_ns()` function [2] [12].
- `cpu_writes(dst_addr, length))` : This routine starts `length` CPU-transfers, with destination address provided as input. The total time required is measured with `timer_get_ticks()` function (through reading of `CNTFRQ_EL0` register, which is a system register that contains the frequency of system timer) and converted into nanoseconds with `timer_ticks_to_ns()` function [2] [12].

In a first step, the variables that significantly influenced transfer times were identified using a **Design of Experiments**. In particular, it was used a **full factorial design with repetitions** where if :

- k : number of factors
- L_i : number of level for factor i
- r : number of repetitions

The number of experiments N that were performed can be calculated as:

$$N = r \cdot \prod_{i=1}^{k} L_i$$

In the case of a two-factor design with repetitions, the response variable is calculated as follows:

$$y_{ijk} = \mu + \alpha_i + \beta_j + (\alpha\beta)_{ij} + \epsilon_{ijk}$$

where:

- y_{ijk} = response variable for level i of factor A, level j of factor B, and repetition k
- μ = overall mean effect
- α_i = main effect of factor A at level i
- β_j = main effect of factor B at level j
- $(\alpha\beta)_{ij}$ = interaction effect between levels i and j of the two factors
- ϵ_{ijk} = random error term for the k-th repetition

To choose which statistical test to use to evaluate the significance of the factors, the residual analysis was performed (normality and homoscedasticity). Subsequently, for the remaining research questions targeted experiments were performed. Each independent trial was repeated multiple times, collecting samples with respect to the number of transfers. Due to the low variance observed within each sample, **mean** $\bar{x}$ was used as central tendency index :

$$\bar{x} = \frac{1}{n} \sum_{i=1}^{n} x_i$$

N.B Only write operations were tested, since they involve three channels (e.g AW,W,B) compared to the two used for read operations (e.g AR,R), thus representing the **worst-case scenario**.

To estimate a model representing the relationship between the dependent variable y (e.g transfer time) and the independent variable x (e.g number of transfers) a linear regression approach was used, when appropriate :

$$\hat{y} = \hat{\beta}_0 + \hat{\beta}_1 x$$

This approach allowed for a systematic comparison between DMA and CPU, with and without FPGA. For evaluating the effects of concurrent accesses, another Jailhouse cell was created, where a periodic task performs a certain number of writes during each period. This setup allows assessing how simultaneous memory operations impact performance and timing.

B. Significant Factors

Initially a design of experiment was conducted to show what factors are considered significant for variable **Time (ns)**. As mentioned in *Metodology* section, it was chosen a full factorial design with 3 repetitions and 2 factors. The factors chosen are :

1) **FPGA:** it can assume only two values , YES or NOT to signal if DMA transfer goes through FPGA or directly in memory
2) **DmaTransfers:** Number of consecutive DMA transfers performed, each of which has a size of 1000 words (called `BURST_LENGH`). The values that this parameter assumes are 5000, 10000, 15000, 20000.

Subsequently independent tests were conducted for each factor combination and residuals for response time Time(ns) were analyzed to determine the statistical test to use. It can be

shown that residuals are normal but not homoscedastic. This suggested using Welch test as statistical test, which showed that **FPGA** is significant with $\alpha = 0.05$, while **DmaTransfers** is not.

C. Impact of FPGA on DMA Transfer Time

Having demonstrated that FPGA is a significant factor, the purpose of this section is to expand the analysis by comparing the time required to transfer data via DMA or Processor to FPGA and directly to memory. In both cases 10 indipendent samples, of 5 elementes, are collected and for each sample the **mean** is considered due to the low COV coefficient.

RQ2: What is the impact of adopting `PAC_MAC` on transfer times of DMA?

the test compared the increase of transfer time as a function of number of DMA transfers, considering both FPGA access and directly memory access, bypassing FPGA. Even in this case each DMA transfer consists of a constant number of words (i.e `BURST_LENGTH=1000`). The following notations are used to discriminate the two cases:

1) **DMA through FPGA to Mem**
2) **DMA to Mem**

In both cases it can be shown that the relationship between the two variables is linear, and the slope of Transfer time of DMA through FPGA to Mem is much larger than Transfer time of `DMA to MEM` [Figure 6]. The assumptions about linear regression are verified in both cases, they have not been reported so as not to weigh down the discussion. The two lines [Figure 6] are described by the following equations:

$$T_{DmaFPGA}(n) = \alpha_{DmaFPGA} + \beta_{DmaFPGA} n \quad (1)$$

$$T_{DmaMem}(n) = \alpha_{DmaMem} + \beta_{DmaMem} n \quad (2)$$

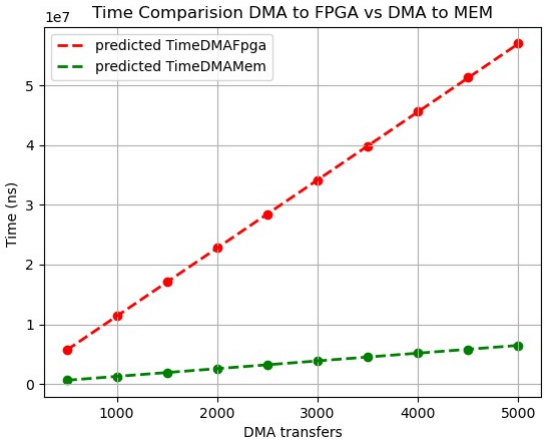


Fig. 6. DMA times comparison without vs with FPGA.

D. Impact of FPGA on CPU Transfer Time

The second test, similar to the first, compares the CPU transfer time on two paths:

1) **CPU through FPGA to MEM**
2) **CPU to MEM**

For the CPU, just as we saw for the DMA, the transfer time along path `CPU through FPGA to MEM` is significantly higher than along path `CPU to MEM` [Figure 7].

$$T_{CpuFPGA}(n) = \alpha_{CpuFPGA} + \beta_{CpuFPGA} n \quad (3)$$

$$T_{CpuMem}(n) = \alpha_{CpuMem} + \beta_{CpuMem} n \quad (4)$$

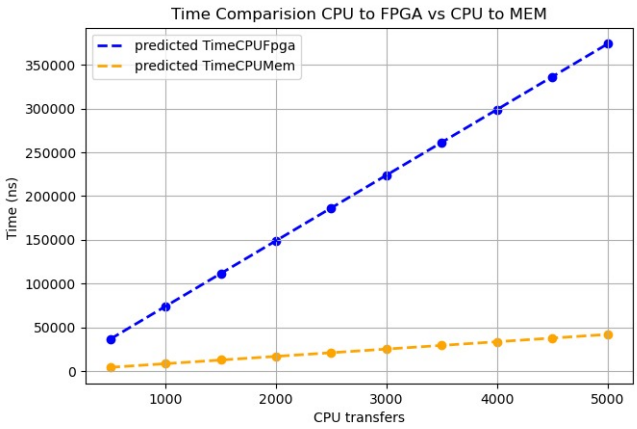


Fig. 7. CPU times comparison without vs with FPGA.

We define in both cases the gain factor **G**:

$$G_{Dma} = \frac{\beta_{DmaFpga}}{\beta_{DmaMem}} \quad (5)$$

$$G_{Cpu} = \frac{\beta_{CpuFpga}}{\beta_{CpuMem}} \quad (6)$$

As shown in the *Result* section, in both cases the gain factor is approximately 9 signaling that Transfer Time Through FPGA increases as function of Number of transfer more rapidly than Transfer Time bypassing FPGA.

E. Cuncurrent Access Analysis

A farther test conducted is the analysis of interference due the presence of another cell (called interfering cell) on which a periodic task with period T (e.g 5ms) performs a certain number of writes to FPGA. Two cases are considered and in both cases 15 samples, of 3 elementes, are collected and the **mean** for each sample is considered due to the low variance from the mean:

1) **DMA transfers on a cell with interfering cell.** On the first cell where DMA transfers are performed (called `dma_cell`) the stress test seen before was launched for `20_000` transfers (each transfer has a `BURST_LENGTH=1000`). On the interfering cell the periodic task was started, gradually increasing the number of transfers performed in each period.

A simple model that describes the relationship between two variables is a linear regression model [Figure 8]. The associated assumptions about normality and homoscedasticity are verified [Figure 9] [Figure 10].

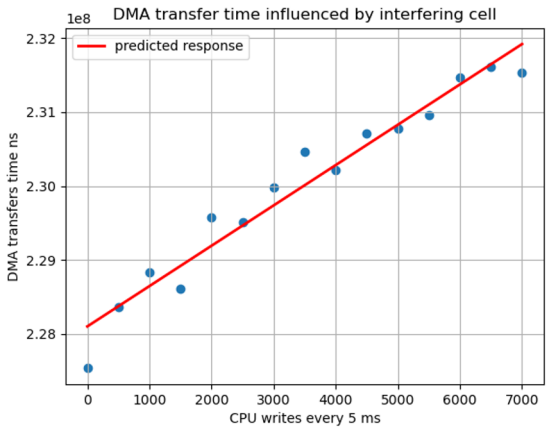


Fig. 8. DMA regressive line.

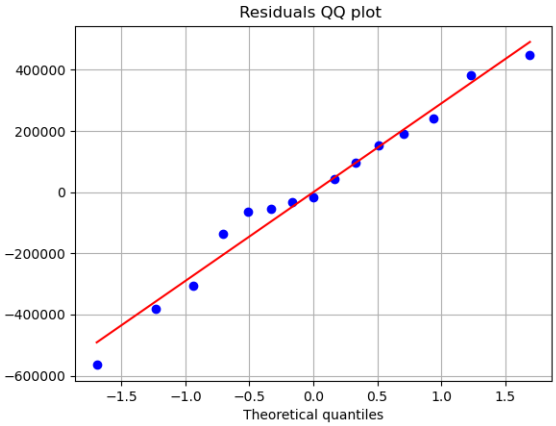


Fig. 9. DMA transfers: residuals QQ plot.

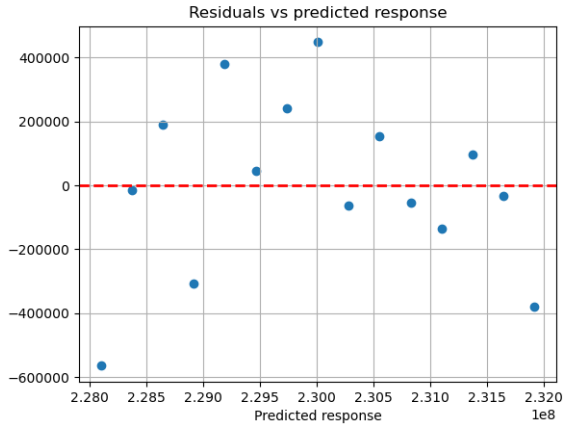


Fig. 10. DMA transfers: residuals homoscedasticity.

2) **CPU transfers on a cell with interfering cell.** On the first cell (called `cpu_cell`) `40_000` CPU transfers are performed. On the interfering cell the periodic task was started, gradually increasing the number of transfers performed in each period.
Here too, a simple model that describes the relationship between two variables is a linear regression model [Figure 11]. The associated assumptions about normality and homoscedasticity are verified also in this case [Figure 12] [Figure 13].

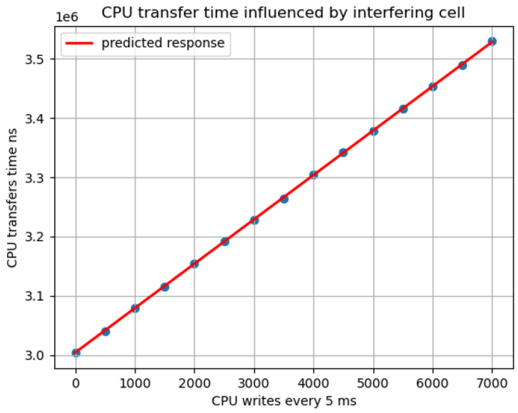


Fig. 11. CPU regressive line.

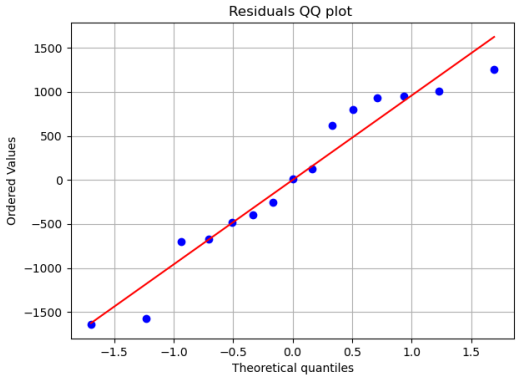


Fig. 12. CPU transfers: residuals QQ plot.

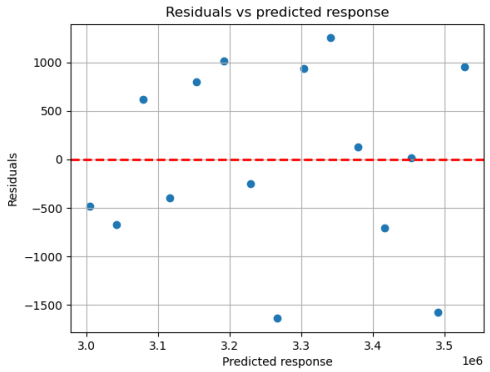


Fig. 13. CPU transfers: residuals homoscedasticity.

F. Results

In this section, we present a summary tables collecting the results of all the experiments. This overview will help to identify common patterns and highlight the most relevant characteristics of the tested scenarios.

TABLE I
DOE RESULTS

	F ratio	Num DF	Den DF	P> $\|t\|$ (Welch Test)
FPGA	42.6652	1	11.284	< 0.0001
DmaTransfers	2.2084	3	9.8705	0.1508

TABLE II
REGRESSION PARAMETERS

	coef	P> $\|t\|$	0.025	0.975
$\alpha_{DmaFPGA}$	-1.126e+04	0.596	-5.83e+04	3.58e+04
$\beta_{DmaFPGA}$	1.14e+04	0.000	1.14e+04	1.14e+04
α_{DmaMem}	-673.6667	0.921	-1.59e+04	1.46e+04
β_{DmaMem}	1296.4421	0.000	1291.520	1301.364
$\alpha_{CpuFPGA}$	-802.0333	0.000	-981.845	-622.222
$\beta_{CpuFPGA}$	74.9368	0.000	74.879	74.995
α_{CpuMem}	270.2333	0.091	-54.067	594.533
β_{CpuMem}	8.3605	0.000	8.256	8.465
G_{Dma}	8.792	–	8.759	8.825
G_{Cpu}	8.964	–	8.854	9.074

X. CONCLUSIONS, LIMITATIONS AND FUTURE WORK

Our work solution solved the security problem represented by full visibility of front-end memory from the back-end. The developed component called PAC-MAC, restricts access only to predefined memory areas (Virtqueues and the two buffer areas). This was achieved through a FILTER sub-module, which checks whether the address of the write/read request is valid and when this is not true, the request is redirected to a SINK sub-module which terminates the master AXI transaction responding with an error response. The statistical analysis shows that FPGA is a significant factor for Time instead of DmaTransfer factor. In this regard, going through FPGA instead of accessing directly in memory with DMA or Processor, causes the time to increase as a function of the number of transfers with a slope 9 times larger (in case of DMA this result is validated when each transfer consists of 1000 words). The proposal described in this work should be considered a first implementation, susceptible to subsequent optimizations and improvements. The analysis has highlighted a significant overhead related to the FPGA transition. This behavior is primarily attributable to the presence of components such as AXI_SMARTCONNECT, which can introduce additional cycles, arbitration delays, and AXI transaction reordering. Other causes of this worsening are:

1) Longer path through FPGA
2) PS Memory Optimizations are no longer valid on PL side
3) Clock domain crossing latency

When there is another VM (or cell in the Jailhouse context) impacting the FPGA, the interference caused by its presence doesn't make the DMA/CPU transfer time much worse.
We have not yet analyzed which is the best solution between `PAC_MAC` approach and existing software solutions but as future work it might be interesting to compare our hardware solution with the software solutions mentioned above (i.e grant tables and bounce buffer) to show which of two is better in terms of overhead. Other future developments could include extending `PAC_MAC` module with additional features such as dynamic filter-range, etc.

REFERENCES

[1] Siemens, Github Repository, "Jailhouse Documentation".
[2] Xilinx, Inc., "Zynq UltraScale+ Device Technical Reference Manual (UG1085)", March 2025.
[3] J. Peixoto, J. Martins, D. Cerdeira, S. Pinto, "BiRtIO: VirtIO for Real-Time Network Interface Sharing on the Bao Hypervisor", Dec 2024.
[4] Xilinx, Inc., "ZCU104 Board User Guide (UG1267)", October 2018.
[5] F. Pizzo, "FPGA-Based Memory Traffic Analysis", June 2023.
[6] F. Pizzo, Github, "Jailhouse on ZCU104", Oct 2024.
[7] ARM, "AXI protocol overview".
[8] Christopher Clark, A Rough Introduction to Using Grant Tables, https://xenbits.xen.org/docs/4.2-testing/misc/grant-tables.txt,March, 2005.
[9] DMA and swiotlb, https://docs.kernel.org/core-api/swiotlb.html
[10] Oasis Open Virtual I/O Device (VIRTIO) Version 1.2 https://docs.oasis-open.org/virtio/virtio/v1.2/virtio-v1.2.html
[11] Eugenio Pérez Martín. Virtqueues and virtio ring: How the data travels https://www.redhat.com/en/blog/virtqueues-and-virtio-ring-how-data-travels
[12] ZDMA Module, https://docs.amd.com/r/en-US/ug1087-zynq-ultrascale-registers/ZDMA-Module
[13] F. Pizzo, Github, "FPGA-Based Memory Traffic Analysis", Oct 2024.

XI. APPENDIX

A. ZCU104 hardware platform

The prototype was implemented on the ZCU104, a device that integrates both programmable logic and a multi-core general-purpose system into a single package, with the following key components:

- **Processing System (PS):** It includes in detail four 64-bit Arm Cortex-A53 cores in the so-called full-power

domain (FPD) and two Cortex-R5 cores in the low-power domain (LPD). These domains have separate clock and reset controllers but share the same address space.
- **Programmable Logic (PL):** The FPGA part, with over 500k logic cells, numerous DSP blocks, and local memory resources. It's connected to the PS via high-bandwidth AXI interfaces.
- **External memory:** The board is equipped with a DDR4 controller (integrated into the PS) that manages a 4 GB DDR, which is also accessible from the programmable logic via the AXI interconnects.
- **High band channels:** several High-Performance (HP) and High-Performance Cache-coherent (HPC) ports are available, which enable direct, high-bandwidth transfers between the PS and PL.

Among the DMA controllers integrated on the board, we chose the Full-Power-Domain DMA (FPD-DMA), a two-channel peripheral that is part of the Full Power Domain (FPD). This means it works closely with the Application Processing Unit (APU) and allows transfers between memory and the PL without going through the Cache Coherent Interconnect (CCI) manager [4].

B. Development Process Guide

Starting from guide [13], it will be possible to set up the development environment (Jailhouse and Vivado). To be able to use the functions employed in this work, the following steps need to be executed starting from our repository:

Jailhouse settings :

- Integrate the **dma_lib** folder into Jailhouse by copying it to the path **/jailhouse/inmates/lib**
- Add the file **dma_utils.o** present in **dma_lib** to the Makefile located in **/jailhouse/inmates/lib/arm64**
- Add to the Makefile located in **/jailhouse/inmate/** (in the `include` section) the command: `-I/$(src)/lib/dma_lib/include`

Once this is done, it will be possible to develop your own code that uses this library and that can then be loaded into a Jailhouse cell. Before launching the Jailhouse compilation, execute the following steps so that the .bin files of the developed source codes are produced:

- Develop your own inmate (e.g., dma_transfer.c) in **/jailhouse/demos/arm**
- Add in the Makefile located in **/jailhouse/demos/arm64** on the `INMATES` line the name of your file in .bin format (e.g., dma_transfer.bin)
- Add in the Makefile located in **/jailhouse/demos/arm64** another line of the type: `dma_transfer.y := ../arm/dma_transfer.o`

Once this is done, launch the compilation and extract the image to load onto the microSD as described in guide [13].
N.B :You may need to force copy the updated Jailhouse folder to your microSD card with command `sudo cp -r path../jailhouse /media/user_name/RootFS`
Then, on the board, enable Jailhouse by loading the kernel module **jailhouse.ko** located in **/jailhouse/driver**, and subsequently create the root cell with the command `sudo jailhouse enable zynqmp-zcu104.cell` and the non-root cell with DMA support using the command `sudo jailhouse enable zynqmp-zcu104-dma.cell`.
Load the desired .bin (e.g., dma_transfer.bin) into the non-root Jailhouse cell with `sudo jailhouse cell load 1 dma_transfer.bin -s "con-type=JAILHOUSE" -a 0x1000`.
Then start the cell (e.g., dma_cell) with `sudo jailhouse cell start dma_cell`

Vivado & Petalinux settings:
Regarding the VIVADO Design, modify the one reported in guide [13] by removing the IP `ILA`.
Go to **Settings** and in the section **IPs** add the repository of the custom IP `PAC_MAC`. Adds the IP `PAC_MAC` in the design, using **Run Connection Automation** to connect the IP to the other components of the Design.
N.B It might be necessary to configure the AXI Smartconnect present in the Design to manually connect the M_AXI interface of `PAC_MAC` IP to **S_AXI_HP0_FPD** port of Processing System and to **S01_AXI** interface of `PAC_MAC`.
Assign in the address editor, for the slave interface S01_AXI of `PAC_MAC` the range [0x11_0000_0000,0x11_7FFF_FFFF] and for slave interface S00_AXI of `PAC_MAC` the range [0x0000_00000,0x7FFF_FFFF]. For the other IPs like ATL follow [13].
N.B. Click **Assign All**, before doing the address assignment, if you see an interface labeled as **Unassign**.
Generate the bitstream and extract the hardware design as a .xsa file. Modify the design in Petalinux with the command `petalinux-config --get-hw-description =<path_to_xsa_file>`
N.B: In some cases it may be necessary to clean the project first with `petalinux -x mrproper` before running the previous command. This restores the project to the state prior to the build, so the steps required to use Jailhouse (e.g patching the Kernel) will need to be repeated [13].
Relaunch the Petalinux build with the command `petalinux-build`, extract the SD image as reported in [13], and then connect the microSD to the board.